Shivam Kumar
Roll. No.: 22CE01046
November 24, 2025
GitHub: https://github.com/shivamkumar

Assignment-2 (CAD Laboratory)

Question 1. Given the two-dimensional body

$$\mathcal{B} = \{(X_1, X_2) | 0.1 < X_1 < 1, 0.1 < X_2 < 1\}$$

and the displacement field

$$u_1 = u \cdot e_1 = 0.2 \ln(1 + X_1 + X_2), u_2 = u \cdot e_2 = 0.2 \exp X_1.$$

Plot the displaced shape of the body using Julia.

Hint: First set up some characteristic lines on the body. For each point compute its displacement and add it to the original position by noting

$$x_i = e_i \cdot (X + u)$$

Solution: 1

```julia
using Plots

# 1. Define the original domain (undeformed grid)
x1_range = 0.1:0.05:1.0
x2_range = 0.1:0.05:1.0

# Lists to store coordinates
X1_orig, X2_orig = Float64[], Float64[]
x1_new, x2_new = Float64[], Float64[]

# 2. Compute displacement and new positions
for x1 in x1_range
    for x2 in x2_range
        # Original Coordinates
        push!(X1_orig, x1)
        push!(X2_orig, x2)

        # Displacement field u1, u2 [cite: 16]
        u1 = 0.2 * log(1 + x1 + x2)
        u2 = 0.2 * exp(x1)

        # Deformed Coordinates xi = Xi + ui
        push!(x1_new, x1 + u1)
        push!(x2_new, x2 + u2)
    end
```

```
        end

        # 3. Plotting
        scatter(X1_orig, X2_orig, label="Original Configuration", marker=:circle,
        ↪  legend=:topleft)
        scatter!(x1_new, x2_new, label="Deformed Configuration", marker=:cross)
        title!("Q1: Displacement of Rectangular Body")
        xlabel!("X1")
        ylabel!("X2")
```

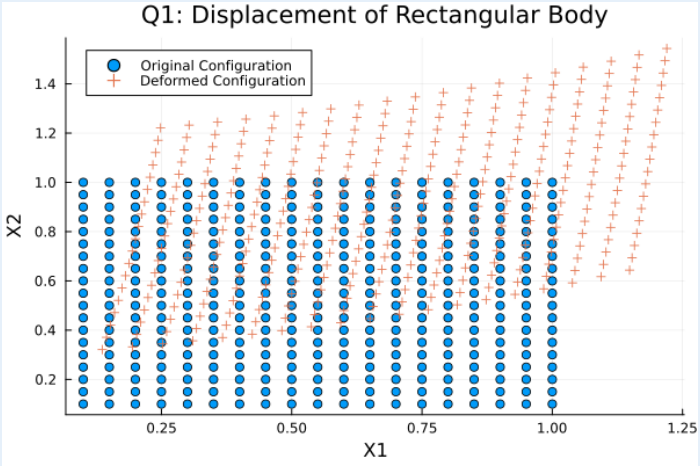

FIGURE 1. Displacement of Rectangular Body

Question 2. With respect to a two-dimensional rectangular cartesian coordinates system with orthonormal base vectors (e_1, e_2), let the rectangular coordinates of a point be denoted by (X_1, X_2). Consider a two-dimensional annular body $\mathcal{B}$ which occupies the region:

$$\mathcal{B} = \{(X_1, X_2) | 1 < \sqrt{X_1^2 + X_2^2} < 2\}$$

.

Because of the annual nature of the body consider a polar coordinates system in which the coordinates of points are expressed in terms (R, θ), which are related to the rectangular coordinates (X_1, X_2) by:

$$R = \sqrt{X_1^2 + X_2^2} and \theta = \tan^{-1}(X_2/X_1)$$

Further, the orthonormal base vectors (e_r, e_θ) of the polar coordinate system are related to (e_1, e_2) by:

$$e_r = \cos\theta e_1 + \sin\theta e_2, e_\theta = -\sin\theta e_1 + \cos\theta e_2$$

Draw the deformed configuration of the annular body associated with the following deformation using Julia:

$$u_r = u \cdot e_r = 0.4(R-1)^2 \cos 3\theta$$

$$u_\theta = u \cdot e_\theta = 0.4(R-1)^3$$

Hint: First set up some characteristic lines on the body in terms of their (R, θ) coordinates. Compute the (X_1, X_2) coordinates from

$$X_1 = R\cos\theta, X_2 = R\sin\theta$$

. For each point compute its displacement and add it to the original position by noting $x_i = e_i \cdot (X + u) = e_i \cdot (X + u_r e_r + u_\theta e_\theta)$.

Solution: 2

```
        using Plots

        # 1. Define the annular domain using Polar Coordinates
        R_range = 1.0:0.1:2.0
        Theta_range = 0:0.01:2*pi

        X1_orig, X2_orig = Float64[], Float64[]
        x1_new, x2_new = Float64[], Float64[]

        for r in R_range
            for theta in Theta_range
                # Convert Polar to Cartesian for Original Points [cite: 33]
                X1 = r * cos(theta)
                X2 = r * sin(theta)

                push!(X1_orig, X1)
                push!(X2_orig, X2)

                # Calculate Displacements in Polar Basis [cite: 29]
                u_r = 0.4 * (r - 1)^2 * cos(3 * theta)
                u_theta = 0.4 * (r - 1)^3

                # Transformation to Cartesian Basis
                # u1 = u_r * cos(theta) - u_theta * sin(theta)
                # u2 = u_r * sin(theta) + u_theta * cos(theta)
                # Derived from basis vector relations [cite: 25, 34]

                u1 = u_r * cos(theta) - u_theta * sin(theta)
                u2 = u_r * sin(theta) + u_theta * cos(theta)

                # Add displacement to original Cartesian coords
                push!(x1_new, X1 + u1)
                push!(x2_new, X2 + u2)
            end
        end

```

```
        # 3. Plotting
        scatter(X1_orig, X2_orig, label="Original", aspect_ratio=:equal, markersize=2)
        scatter!(x1_new, x2_new, label="Deformed", markersize=2)
        title!("Q2: Annular Body Deformation")

        savefig("Annular_Body_Deformation.png")    # save the figure
```

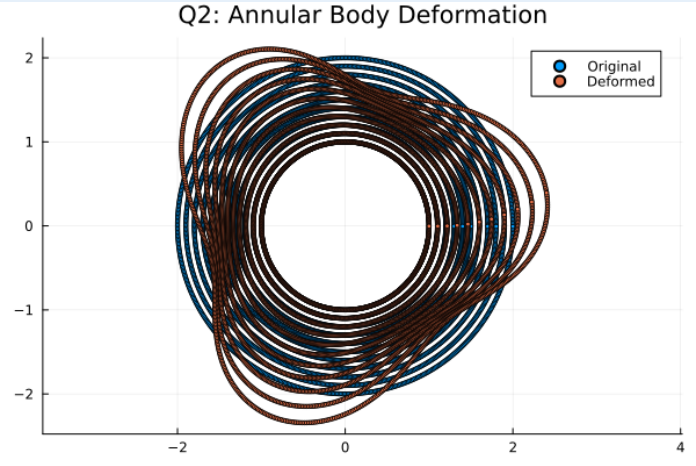

FIGURE 2. Annular Body Deformation

Question 3. Given the two-dimensional body

$$\mathcal{B} = \{(X_1, X_2) | 0.1 < X_1 < 1, 0.1 < X_2 < 1\}$$

,

and the displacement field

$$u_r = u \cdot e_r = 0.2 \exp X_1, u_\theta = u \cdot e_\theta = 0.2 \ln(1 + X_1 + X_2)$$

.

Plot the displaced shape of the body using Julia.

Solution: 3

```
        using Plots

        x1_range = 0.1:0.05:1.0
        x2_range = 0.1:0.05:1.0

        X1_orig, X2_orig = Float64[], Float64[]
        x1_new, x2_new = Float64[], Float64[]

        for X1 in x1_range
            for X2 in x2_range
                push!(X1_orig, X1)
```

```
                push!(X2_orig, X2)

                # Calculate local polar parameters for basis transformation [cite: 23]
                R = sqrt(X1^2 + X2^2)
                theta = atan(X2, X1)

                # Calculate displacements (given values) [cite: 38, 40]
                u_r_val = 0.2 * exp(X1)
                u_theta_val = 0.2 * log(1 + X1 + X2)

                # Convert to Cartesian components u1, u2 using local theta
                # u = u_r * e_r + u_theta * e_theta
                u1 = u_r_val * cos(theta) - u_theta_val * sin(theta)
                u2 = u_r_val * sin(theta) + u_theta_val * cos(theta)

                push!(x1_new, X1 + u1)
                push!(x2_new, X2 + u2)
            end
        end

        scatter(X1_orig, X2_orig, label="Original", legend=:topleft)
        scatter!(x1_new, x2_new, label="Deformed")
        title!("Q3: Mixed Basis Deformation")

        savefig("Mixed_Basis_Deformation.png")    # save the figure
```

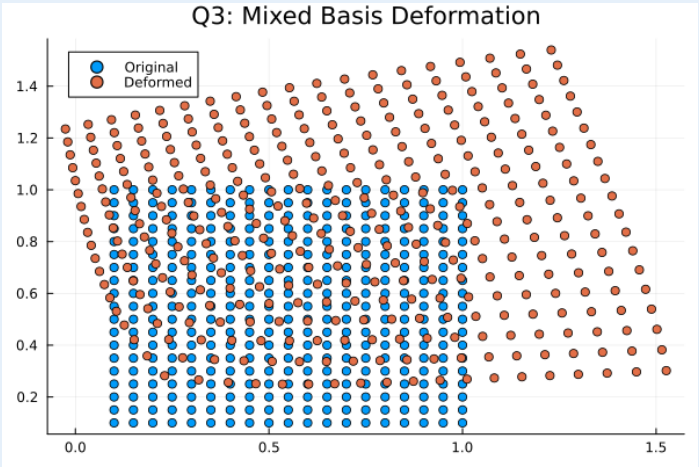

FIGURE 3. Mixed Basis Deformation

Question 4. Find out the stress distribution by modeling the given rectangular plate of 2 mm thick with a circular hole. The dimensions of the steel plate and hole are shown in Figure 1. The section is made up of steel having a value of 210 GPa for Young's modulus and a value of 0.3 for Poisson's ratio.

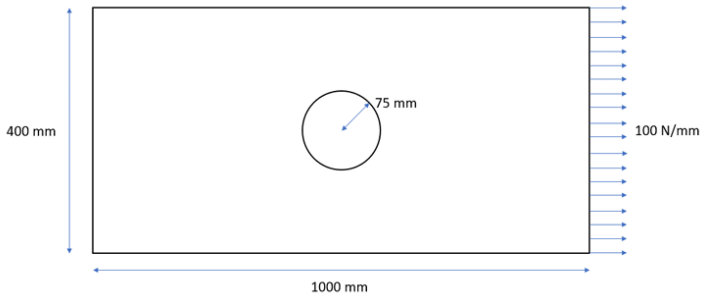

FIGURE 4. Caption

(a) Provide the steps for finite element meshing both in Julia and Abaqus.
(b) Mention the governing PDEs that are required to solve for the stress distribution.
(c) Provide a comparison between the results obtained from Gridap in Julia and Abaqus.

Solution: 4 (a) Steps for finite element meshing using Julia.

```
        import Gmsh: gmsh

        # Initializing Gmsh
        gmsh.initialize()

        gmsh.model.add("PlateHole_Q4")

        # --- Parameters (Converted to Meters to match your L=1 scale) ---
        L = 1.0      # 1000 mm
        H = 0.4      # 400 mm
        R = 0.075    # Radius 75 mm
        Xc = 0.5     # Center X (500 mm)
        Yc = 0.2     # Center Y (200 mm)

        # Mesh sizes
        h_coarse = 0.008  # Coarse mesh for far boundaries
        h_fine   = 0.005 # Fine mesh for the hole (CRITICAL for stress concentration)

        # --- 1. Define the Outer Rectangle ---
        p1 = gmsh.model.geo.addPoint(0, 0, 0, h_coarse)
        p2 = gmsh.model.geo.addPoint(L, 0, 0, h_coarse)
        p3 = gmsh.model.geo.addPoint(L, H, 0, h_coarse)
        p4 = gmsh.model.geo.addPoint(0, H, 0, h_coarse)

```

```
        l1 = gmsh.model.geo.addLine(p1, p2)
        l2 = gmsh.model.geo.addLine(p2, p3)
        l3 = gmsh.model.geo.addLine(p3, p4)
        l4 = gmsh.model.geo.addLine(p4, p1)

        # Outer Loop
        cl_rect = gmsh.model.geo.addCurveLoop([l1, l2, l3, l4])

        # --- 2. Define the Inner Circle ---
        # We create 5 points: Center + 4 points on the circumference (Top, Right, Bottom,
        ↪  Left)
        # This method (Circle Arcs) is more robust for meshing than a single full
        ↪  circle.

        pc = gmsh.model.geo.addPoint(Xc, Yc, 0, h_fine) # Center
        p_r = gmsh.model.geo.addPoint(Xc + R, Yc, 0, h_fine)
        p_t = gmsh.model.geo.addPoint(Xc, Yc + R, 0, h_fine)
        p_l = gmsh.model.geo.addPoint(Xc - R, Yc, 0, h_fine)
        p_b = gmsh.model.geo.addPoint(Xc, Yc - R, 0, h_fine)

        # Create 4 arcs to form the circle
        c1 = gmsh.model.geo.addCircleArc(p_r, pc, p_t)
        c2 = gmsh.model.geo.addCircleArc(p_t, pc, p_l)
        c3 = gmsh.model.geo.addCircleArc(p_l, pc, p_b)
        c4 = gmsh.model.geo.addCircleArc(p_b, pc, p_r)

        # Inner Loop
        cl_hole = gmsh.model.geo.addCurveLoop([c1, c2, c3, c4])

        # --- 3. Create Surface with Hole ---
        # The first loop is the boundary, subsequent loops are holes
        s = gmsh.model.geo.addPlaneSurface([cl_rect, cl_hole])

        # --- 4. Physical Groups (Boundary Conditions) ---
        # Domain
        gmsh.model.addPhysicalGroup(2, [s], 1, "Plate")

        # Boundaries
        gmsh.model.addPhysicalGroup(1, [l4], 2, "FixedLeft")  # Left Edge
        gmsh.model.addPhysicalGroup(1, [l2], 3, "LoadRight")  # Right Edge
        gmsh.model.addPhysicalGroup(1, [c1, c2, c3, c4], 4, "HoleEdge") # Hole boundary

        # Synchronize and Mesh
```

```
        gmsh.model.geo.synchronize()
        gmsh.model.mesh.generate(2)

        # Write mesh to file
        gmsh.write("PlateHole_Q4.msh")

        # Launch GUI to see the result
        gmsh.fltk.run()
        gmsh.finalize()
```

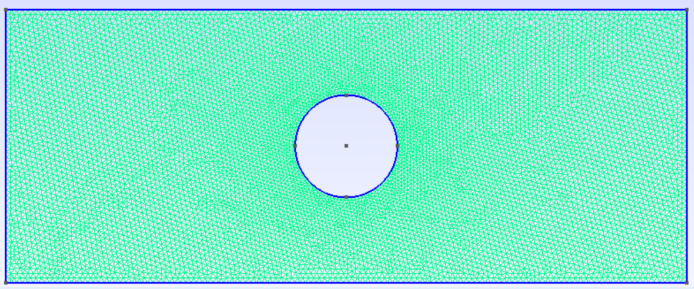

FIGURE 5. Finite element meshing using Julia.

4(a) Steps for finite element meshing using Abaqus.

1) Create a 2D deformable part
   - Start Abaqus/CAE → go to the Part module.
   - Create a new part:
     - Name: PlateHole_Q4.
     - Modeling space: 2D planar.
     - Type: Deformable.
     - Base feature: Shell (plane stress) or Planar.
     - Approximate size: slightly larger than 2000 mm.
2) Sketch the plate geometry
   - In the sketcher, draw a rectangle representing the plate:
     - One corner at (0,0).
     - Opposite corner at (1000,400) mm (length × height).
   - In the same sketch, create a circle with:
     - Center at (500,200) mm and Radius = 75 mm.
   - Finish the sketch to form the final 2D plate-with-hole geometry.
3) Define material and section (for completeness)
   - Go to the Property module.
   - Create a material:
     - Name: Steel.
     - Elastic properties: Young's modulus E=210 GPa Poisson's ratio ν =0.3.
   - Create a solid/plane stress section: Thickness = 2 mm.

- Assign this section to the entire plate part.

4) Create assembly
   - Move to the Assembly module.
   - Create a dependent instance of the part (Create Instance).
5) Choose element type
   - Go to the Mesh module.
   - Assign element type:
     - Element family: plane stress (e.g., CPS4/CPS4R).
     - Suitable for 2D stress analysis of thin plate.
6) Seed the edges (global and local refinement)
   - Apply a global seed size (coarse mesh) for the plate
   - Apply a smaller seed size on the hole boundary
7) Generate the mesh
   - With seeding done, Abaqus will generate the 2D finite element mesh for the plate with a hole

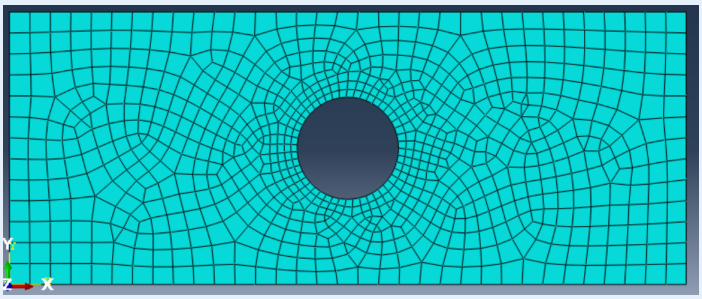

FIGURE 6. Finite element meshing using Abaqus

Solution: **4** (b) Stress distribution.

```
using Gridap
using GridapGmsh

# 1. Read the Mesh
model = GmshDiscreteModel("PlateHole_Q4.msh")

# 2. Material Parameters (Steel)
const E = 210000.0  # MPa
const ν = 0.3       # Poisson's ratio
const thickness = 2.0 # mm

# Lamé parameters for Plane Stress
const E = 210000.0                              # N/mm^2
const ν = 0.3                                   # Poissons Ratio

const μ = E/(2*(1+ν))
```

```
        const λ = (E * ν) / ((1 + ν) * (1 - 2 * ν))
        const λ_ps = (2 * λ * μ) / (λ + 2 * μ)          # Plane stress correction

        g = VectorValue(100.0, 0.0)                 # Force

        σ(ε) = λ_ps*tr(ε)*one(ε) + 2*μ*ε

        # 3. Define FE Spaces
        order = 1

        reffe = ReferenceFE(lagrangian,VectorValue{2,Float64},order)
        V0 = TestFESpace(model,reffe;
            conformity=:H1,
            dirichlet_tags=["FixedLeft"],
            dirichlet_masks=[(true,true)])

        g1 = VectorValue(0.0,0.0)
        U = TrialFESpace(V0,[g1])

        # 4. Numerical Integration
        degree = 2*order
        Ω = Triangulation(model)
        dΩ = Measure(Ω,degree)
        Γ_load  = BoundaryTriangulation(model, tags = "LoadRight")
        dΓ_N = Measure(Γ_load,degree)

        # 5. Define Weak Form

        # Internal Work (Stiffness)
        a(u,v) = ∫( ε(v) ⊙ (σ∘ε(u)) )*dΩ

        # External Work (Load)
        l(v) = ∫(v⋅g)*dΓ_N

        # 6. Solve
        op = AffineFEOperator(a,l,U,V0)
        uh = solve(op)

        # 7. Post-Process
        writevtk(Ω,"results_Q4",
            cellfields=[
                "Displacement"=>uh,
                "Strain"=>ε(uh),
```

```
                "Stress"=>σ∘ε(uh)]
                )

```

4(c) Comparison between the results obtained from Gridap in Julia and Abaqus.

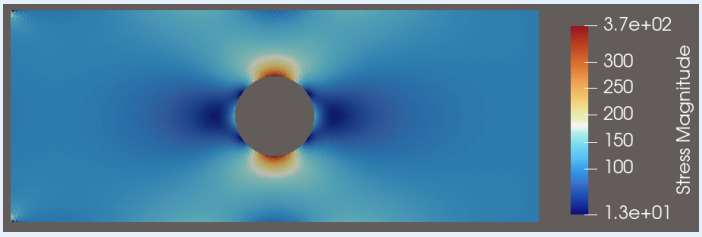

FIGURE 7. Stress distribution using Julia.

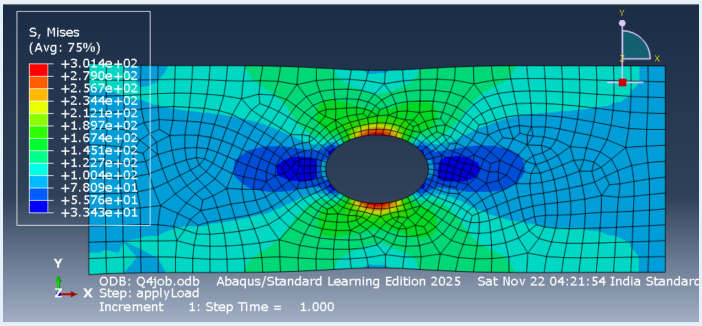

FIGURE 8. Stress distribution using Julia.

Question 5. Find out the stress distribution of the cantilever beam of 1000 mm in length and has a 250 mm × 200 mm rectangular cross-section under the point load of 1000 N at the free end as shown in Figure 2. The section is made up of a concrete having a value of 25000 N/mm² for Young's modulus and a value of 0.2 for Poisson's ratio in the respective fields.

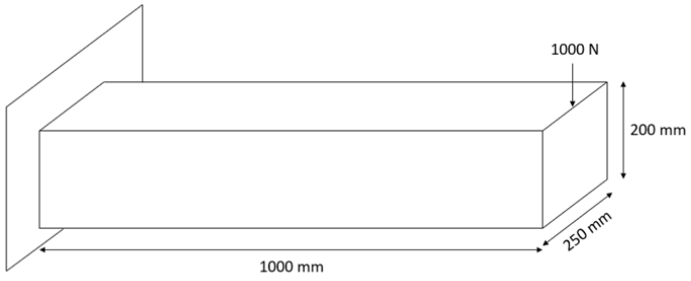

FIGURE 9. Caption

(a) Provide the steps for finite element meshing both in Julia and Abaqus.
(b) Mention the governing PDEs that are required to solve for the stress distribution.
(c) Provide a comparison between the results obtained from Gridap in Julia and Abaqus.

Solution: 5 (a) Steps for finite element meshing using Julia.

```
using Gmsh: gmsh

# Dimensions
L = 1000.0   # x-direction (length)
W = 250.0    # y-direction (width)
H = 200.0    # z-direction (height)
h = 10.0     # mesh size

gmsh.initialize()
gmsh.model.add("Cantilever_Q5")

# --- Create the 8 corner points of the box ---
p1 = gmsh.model.geo.addPoint(0, 0, 0, h)
p2 = gmsh.model.geo.addPoint(L, 0, 0, h)
p3 = gmsh.model.geo.addPoint(L, W, 0, h)
p4 = gmsh.model.geo.addPoint(0, W, 0, h)

p5 = gmsh.model.geo.addPoint(0, 0, H, h)
p6 = gmsh.model.geo.addPoint(L, 0, H, h)
p7 = gmsh.model.geo.addPoint(L, W, H, h)
p8 = gmsh.model.geo.addPoint(0, W, H, h)

# --- Create lines for bottom face ---
l1 = gmsh.model.geo.addLine(p1, p2)
l2 = gmsh.model.geo.addLine(p2, p3)
l3 = gmsh.model.geo.addLine(p3, p4)
l4 = gmsh.model.geo.addLine(p4, p1)

# --- Create lines for top face ---
l5 = gmsh.model.geo.addLine(p5, p6)
l6 = gmsh.model.geo.addLine(p6, p7)
l7 = gmsh.model.geo.addLine(p7, p8)
l8 = gmsh.model.geo.addLine(p8, p5)

# --- Side edges ---
l9  = gmsh.model.geo.addLine(p1, p5)
l10 = gmsh.model.geo.addLine(p2, p6)
l11 = gmsh.model.geo.addLine(p3, p7)
l12 = gmsh.model.geo.addLine(p4, p8)

# ---- Create surfaces (6 faces of the box) ----

```

```
        # Bottom rectangle
        cl1 = gmsh.model.geo.addCurveLoop([l1, l2, l3, l4])
        s1  = gmsh.model.geo.addPlaneSurface([cl1])

        # Top rectangle
        cl2 = gmsh.model.geo.addCurveLoop([l5, l6, l7, l8])
        s2  = gmsh.model.geo.addPlaneSurface([cl2])

        # Side faces
        cl3 = gmsh.model.geo.addCurveLoop([l1, l10, -l5, -l9])
        s3  = gmsh.model.geo.addPlaneSurface([cl3])

        cl4 = gmsh.model.geo.addCurveLoop([l2, l11, -l6, -l10])
        s4  = gmsh.model.geo.addPlaneSurface([cl4])

        cl5 = gmsh.model.geo.addCurveLoop([l3, l12, -l7, -l11])
        s5  = gmsh.model.geo.addPlaneSurface([cl5])

        cl6 = gmsh.model.geo.addCurveLoop([l4, l9, -l8, -l12])
        s6  = gmsh.model.geo.addPlaneSurface([cl6])

        # ---- Create 3D volume ----
        sl = gmsh.model.geo.addSurfaceLoop([s1, s2, s3, s4, s5, s6])
        vol = gmsh.model.geo.addVolume([sl])

        gmsh.model.geo.synchronize()

        # ---- Physical groups ----

        # 3D domain
        gmsh.model.addPhysicalGroup(3, [vol], 10, "Domain")

        # Fixed End (x = 0 → face s6)
        gmsh.model.addPhysicalGroup(2, [s6], 2)
        gmsh.model.setPhysicalName(2, 2, "FixedEnd")

        # Load End (x = L → face s4)
        gmsh.model.addPhysicalGroup(2, [s4], 3)
        gmsh.model.setPhysicalName(2, 3, "LoadFace")

        # ---- Mesh ----
        gmsh.model.mesh.generate(3)

```

```
        # Export mesh
        gmsh.write("Cantilever_Q5.msh")

        # View mesh
        gmsh.fltk.run()
        gmsh.finalize()
```

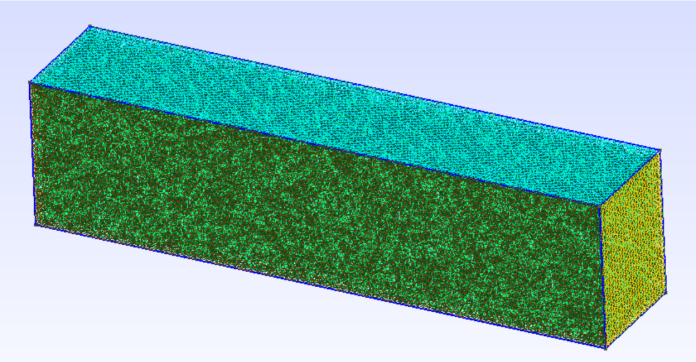

FIGURE 10. Finite element meshing using Julia.

5(a) Steps for finite element meshing using Abaqus.

1) Create a 3D deformable solid part
   - Start Abaqus/CAE → go to the Part module.
   - Create a new part:
     - Name: Cantilever_Q5.
     - Modeling space: 3D.
     - Type: Deformable.
     - Base feature: Solid, Extrusion (or Solid).
     - Approximate size: slightly larger than 2000 mm.
2) Sketch the cross-section and define the beam length
   - In the sketcher, draw a rectangle representing the cross-section:
     - Width = 250 mm, Height = 200 mm.
     - Finish the sketch.
     - When prompted for extrusion: Extrusion length = 1000 mm (this is the beam length).
3) Define material and solid section
   - Go to the Property module.
   - Create a material:
     - Name: Concrete.
     - Elastic properties: Young's modulus E=25000 N/mm2, Poisson's ratio ν =0.2.
   - Create a solid section (e.g. "BeamSection") and assign the material Concrete.
   - Assign this section to the entire beam part.
4) Create assembly
   - Move to the Assembly module.
   - Create a dependent instance of the part (Create Instance).

5) Choose element type and seed the part.
   - Go to the Mesh module.
   - Assign element type:
     – Choose 3D stress elements, e.g. C3D8 or C3D8R (8-node linear brick, reduced integration).
     – Confirm and apply to the solid region.
     – Choose a suitable global seed size along the beam length and across the cross-section.
6) Generate the mesh
   - With seeding done, Abaqus generates the 3D finite element mesh for the cantilever beam.

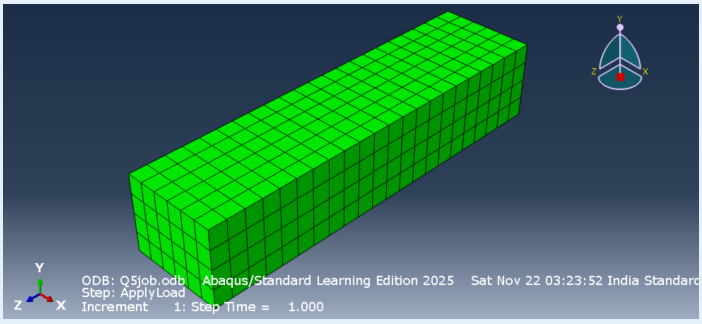

FIGURE 11. Finite element meshing using Abaqus.

Solution: 5 (b) Stress distribution.

```
using Gridap
using GridapGmsh
# 1. Read the Mesh
model = GmshDiscreteModel("Cantilever_Q5.msh")

# 2. Material Parameters (Steel)
const E = 25000.0  # MPa
const ν = 0.2       # Poisson's ratio

# Lamé parameters for Plane Stress

g = VectorValue(0.0,0.0,-1000.0/(250*200)) ## force

const λ = (E*ν)/((1+ν)*(1-2*ν))
const μ = E/(2*(1+ν))
σ(ε) = λ*tr(ε)*one(ε) + 2*μ*ε

# 3. Define FE Spaces
order = 1

reffe = ReferenceFE(lagrangian,VectorValue{3,Float64},order)
V0 = TestFESpace(model,reffe;
```

```
            conformity=:H1,
            dirichlet_tags=["FixedEnd"],
            dirichlet_masks=[(true,true,true)])

        g1 = VectorValue(0.0,0.0,0.0)
        U = TrialFESpace(V0,[g1])

        # 4. Numerical Integration
        degree = 2*order
        Ω = Triangulation(model)
        dΩ = Measure(Ω,degree)
        Γ_load  = BoundaryTriangulation(model, tags = "LoadFace")
        dΓ_N = Measure(Γ_load,degree)

        # Weak from

        # Internal work (Stiffness component)
        a(u,v) = ∫( ε(v) ⊙ (σ∘ε(u)) )*dΩ

        # External Work
        l(v) = ∫(v⋅g)*dΓ_N

        # Solve
        op = AffineFEOperator(a,l,U,V0)
        uh = solve(op)

        # 7. Post-Process
        writevtk(Ω,"results_Q5",
            cellfields=[
                "Displacement"=>uh,
                "Strain"=>ε(uh),
                "Stress"=>σ∘ε(uh)]
                )

```

5(c) Comparison between the results obtained from Gridap in Julia and Abaqus.

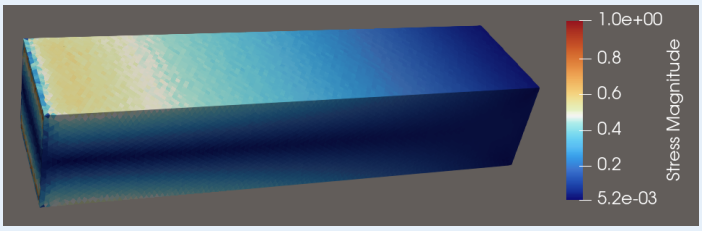

FIGURE 12. Stress distribution using julia

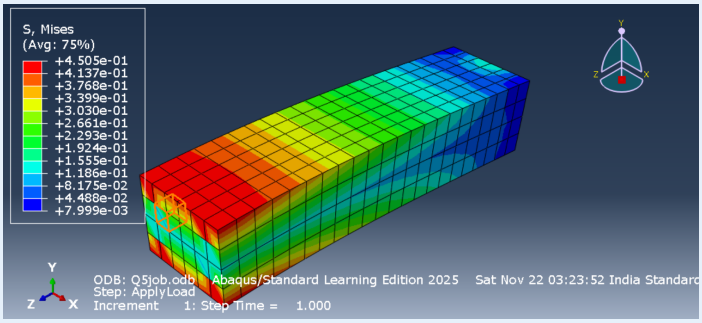

FIGURE 13. Stress distribution using Abaqus

Question 6. Find out the stress distribution of the cantilever beam of 1000 mm in length and has a 250 mm × 200 mm rectangular cross-section under the uniform distributed load of 1000 N/mm² applied on the top surface as shown in Figure 3. The section is made up of a concrete having a value of 25000 N/mm² for Young's modulus and a value of 0.2 for Poisson's ratio in the respective fields.

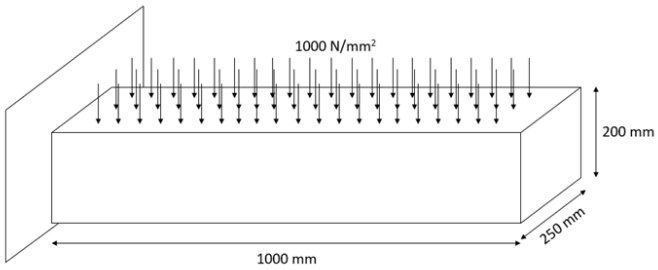

FIGURE 14. Caption

(a) Provide the steps for finite element meshing both in Julia and Abaqus.
(b) Mention the governing PDEs that are required to solve for the stress distribution.
(c) Provide a comparison between the results obtained from Gridap in Julia and Abaqus.

Solution: 6 (a) Steps for finite element meshing using Julia.

```julia
using Gmsh: gmsh

# Dimensions
L = 1000.0    # x-direction (length)
W = 250.0     # y-direction (width)
H = 200.0     # z-direction (height)
h = 10.0      # mesh size

gmsh.initialize()
gmsh.model.add("Cantilever_Q6")

# --- Create the 8 corner points of the box ---
p1 = gmsh.model.geo.addPoint(0, 0, 0, h)
p2 = gmsh.model.geo.addPoint(L, 0, 0, h)
```

```
        p3 = gmsh.model.geo.addPoint(L, W, 0, h)
        p4 = gmsh.model.geo.addPoint(0, W, 0, h)

        p5 = gmsh.model.geo.addPoint(0, 0, H, h)
        p6 = gmsh.model.geo.addPoint(L, 0, H, h)
        p7 = gmsh.model.geo.addPoint(L, W, H, h)
        p8 = gmsh.model.geo.addPoint(0, W, H, h)

        # --- Create lines for bottom face ---
        l1 = gmsh.model.geo.addLine(p1, p2)
        l2 = gmsh.model.geo.addLine(p2, p3)
        l3 = gmsh.model.geo.addLine(p3, p4)
        l4 = gmsh.model.geo.addLine(p4, p1)

        # --- Create lines for top face ---
        l5 = gmsh.model.geo.addLine(p5, p6)
        l6 = gmsh.model.geo.addLine(p6, p7)
        l7 = gmsh.model.geo.addLine(p7, p8)
        l8 = gmsh.model.geo.addLine(p8, p5)

        # --- Side edges ---
        l9  = gmsh.model.geo.addLine(p1, p5)
        l10 = gmsh.model.geo.addLine(p2, p6)
        l11 = gmsh.model.geo.addLine(p3, p7)
        l12 = gmsh.model.geo.addLine(p4, p8)

        # ---- Create surfaces (6 faces of the box) ----

        # Bottom rectangle
        cl1 = gmsh.model.geo.addCurveLoop([l1, l2, l3, l4])
        s1  = gmsh.model.geo.addPlaneSurface([cl1])

        # Top rectangle
        cl2 = gmsh.model.geo.addCurveLoop([l5, l6, l7, l8])
        s2  = gmsh.model.geo.addPlaneSurface([cl2])

        # Side faces
        cl3 = gmsh.model.geo.addCurveLoop([l1, l10, -l5, -l9])
        s3  = gmsh.model.geo.addPlaneSurface([cl3])

        cl4 = gmsh.model.geo.addCurveLoop([l2, l11, -l6, -l10])
        s4  = gmsh.model.geo.addPlaneSurface([cl4])

```

```
        cl5 = gmsh.model.geo.addCurveLoop([l3, l12, -l7, -l11])
        s5  = gmsh.model.geo.addPlaneSurface([cl5])

        cl6 = gmsh.model.geo.addCurveLoop([l4, l9, -l8, -l12])
        s6  = gmsh.model.geo.addPlaneSurface([cl6])

        # ---- Create 3D volume ----
        sl = gmsh.model.geo.addSurfaceLoop([s1, s2, s3, s4, s5, s6])
        vol = gmsh.model.geo.addVolume([sl])

        gmsh.model.geo.synchronize()

        # ---- Physical groups ----

        # 3D domain
        gmsh.model.addPhysicalGroup(3, [vol], 10, "Domain")

        # Fixed End (x = 0 → face s6)
        gmsh.model.addPhysicalGroup(2, [s6], 2)
        gmsh.model.setPhysicalName(2, 2, "FixedEnd")

        # Load End (x = L → face s5)
        gmsh.model.addPhysicalGroup(2, [s5], 3)
        gmsh.model.setPhysicalName(2, 3, "LoadFace")

        # ---- Mesh ----
        gmsh.model.mesh.generate(3)

        # Export mesh
        gmsh.write("Cantilever_Q6.msh")

        # View mesh
        gmsh.fltk.run()
        gmsh.finalize()

```

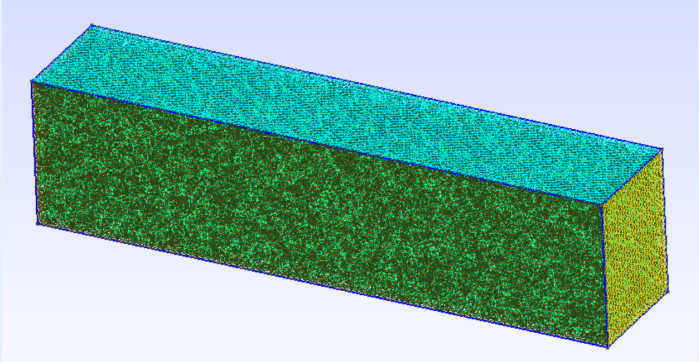

FIGURE 15. Finite element meshing using Julia.

6(a) Steps for finite element meshing using Abaqus.

- Same step as follow in 5.

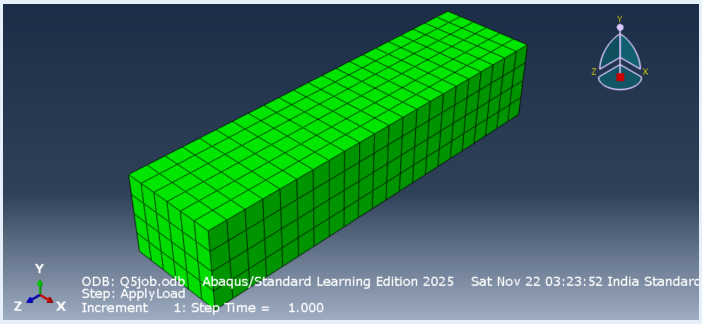

FIGURE 16. Finite element meshing using Abaqus.

Solution: 6 (b) Stress distribution.

```
using Gridap
using GridapGmsh

# 1. Read the Mesh
model = GmshDiscreteModel("Cantilever_Q6.msh")

# 2. Material Parameters (Steel)
const E = 25000.0  # MPa
const ν = 0.2       # Poisson's ratio

# Lamé parameters for Plane Stress

g = VectorValue(0.0,0.0,-1000.0)     ## force

const λ = (E*ν)/((1+ν)*(1-2*ν))
const μ = E/(2*(1+ν))
σ(ε) = λ*tr(ε)*one(ε) + 2*μ*ε

```

```
        # 3. Define FE Spaces
        order = 1

        reffe = ReferenceFE(lagrangian,VectorValue{3,Float64},order)
        V0 = TestFESpace(model,reffe;
            conformity=:H1,
            dirichlet_tags=["FixedEnd"],
            dirichlet_masks=[(true,true,true)])

        g1 = VectorValue(0.0,0.0,0.0)
        U = TrialFESpace(V0,[g1])

        # 4. Numerical Integration
        degree = 2*order
        Ω = Triangulation(model)
        dΩ = Measure(Ω,degree)
        Γ_load  = BoundaryTriangulation(model, tags = "LoadFace")
        dΓ_N = Measure(Γ_load,degree)

        # Weak from

        # Internal work (Stiffness component)
        a(u,v) = ∫( ε(v) ⊙ (σ∘ε(u)) )*dΩ

        # External Work
        l(v) = ∫(v⋅g)*dΓ_N

        # Solve
        op = AffineFEOperator(a,l,U,V0)
        uh = solve(op)

        # 7. Post-Process
        writevtk(Ω,"results_Q6",
            cellfields=[
                "Displacement"=>uh,
                "Strain"=>ε(uh),
                "Stress"=>σ∘ε(uh)]
                )
```

6(c) Comparison between the results obtained from Gridap in Julia and Abaqus.

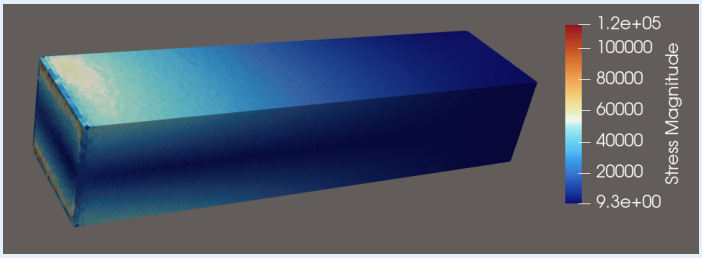

FIGURE 17. Stress distribution using julia

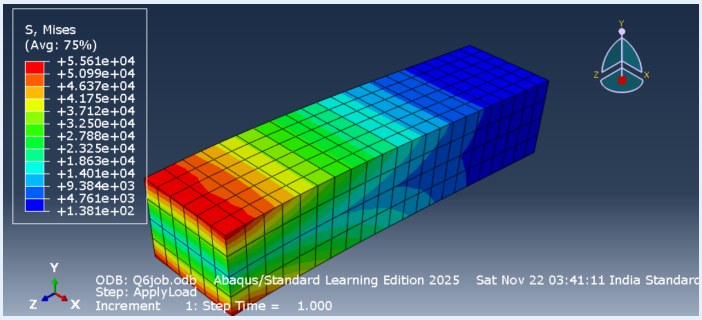

FIGURE 18. Stress distribution using Abaqus

– – – The End – – –